

YIĞIT VERİ YAPISI

Dr. Öğr. Üyesi Fatma AKALIN

Bilgisayar bilimlerindeki en kullanışlı kavramlardan birisi Yığıt veri yapısıdır. **Yığıt** veri yapısı tanımı yönüyle oldukça basit olmasıyla birlikte programlama ve programlama dillerinde çok önemli bir yere sahiptir.

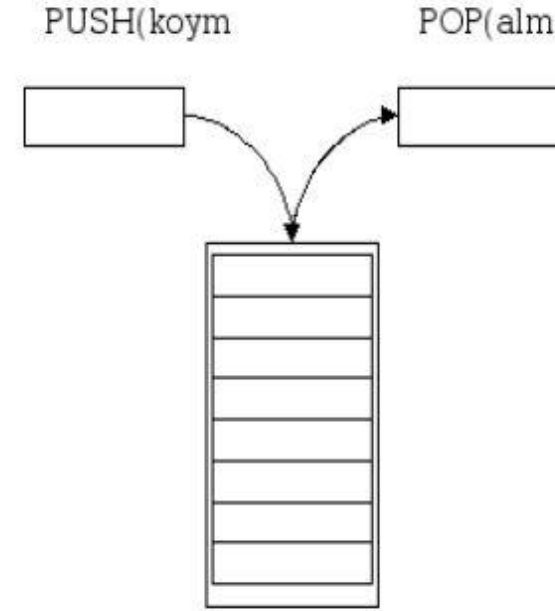
Diziler konusundan hatırlanacağı gibi tek boyut ve çok boyutlu dizilerin belleğe erişimleri oldukça kolaydır. Çünkü diziler bellekte art arda gelen hücrelerde saklanmaktadır ve bu hücrelerden ilkinin adresi biliniyorsa herhangi bir verinin adresi basit bir hesap ile bulunabilmektedir. Dolayısıyla herhangi bir anda herhangi bir verinin adresine ulaşılması bir adres polinomu hesabı ile mümkündür. Dizideki eleman sayısı ne olursa olsun herhangi bir elemana ulaşmanın algoritma karmaşıklığı $O(1)$ 'dir. Fakat birçok problemin doğası elemanlarına sadece bir noktadan belirli bir sıraya göre erişimi gerektirebilir. Bu tür uygulamalarda dizilerde olduğu gibi herhangi bir elemana istediğimiz anda ulaşmamız mümkün değildir. Ayrıca yeni bir eleman eklemek istediğimizde de bunu herhangi bir yere yerleştiremeyiz. Örneğin Word belgesinde geri tuşuna bastığımızda uyulması gereken bir sıra vardır.

Dizi ile Baęlı Liste Arasındaki Farklar

Dizi (Array)	Baęlı Liste (Linked List)
Dizilerin belirli bir kapasitesi vardır. (Program başladıęında belirlenir)	Baęlı listelerin bir kapasitesi yoktur belleęin izin verdięi miktarda büyüeyebilirler.
Diziler birbirinin arkasına sıralı bir şekilde gelir. (Dizi[0], Dizi[1] den 4 bayt (integer dizi için) sonradır)	Baęlı listedeki elemanlar bellekte herhangi bir yerde olabilirler.
Bir diziye eleman eklemek ve çıkartmak çok maliyetlidir. (5 elemanlık bir diziye 6. elemanı eklemek isterseniz ilk önce önceki 5 elemanlık diziyi bellekte başka bir yere taşımamız gerekir)	Baęlı listelerde eleman eklemek ve çıkarmak nispeten daha kolaydır. (Sadece en son düęümü veya silinecek düęümü bulmak gerekir)
Dizilerde arama çok daha kolaydır. Çünkü bellekte ardışık yer kaplarlar. (Algoritma Karmaşıklığı $O(1)$)	Baęlı listelerde arama yapmak zordur çünkü bir düęüme ulaşmak için önceki düęümlerden gelmeniz gerekir. (Algoritma Karmaşıklığı $O(n)$)
Diziler bellekte daha az yer kaplar (Sadece değerleri tutarlar)	Baęlı listeler bellekte daha fazla yer kaplar (Hem değerleri hem de sonraki düęümün adresini tutarlar)

<https://www.ysancar.com/veri-yapilari/dizi-ile-bagli-liste-arasindaki-farklar/>

Yandaki resimde görüldüğü gibi
eleman ekleme ve çıkarma
işlemlerinin dizinin sadece bir
ucundan yapılabildiği veri saklama
sistemlerine yığıt denir.



YIĞIN VERİ YAPISI (STACK) İŞLEM ADIMLARI VE GÖRSELLEŞTİRMESİ

Adım 1: Başlangıç ve Ekleme (Push)	Adım 2: Veriler Eklendi	Adım 3: En Üstten Veri Çekme (Pop)	Adım 4: Yeni Veri Ekleme
Boş Yiğit koy(5); koy(70); koy(99); yi → 99 70 5 Önce 5, sonra 70, en son 99 eklenir.	99 en üste yerleşir. yi → 99 70 5 En son giren 99, en üsttedir.	k = al() \$k = 99 Veri Alındı (Pop) yi → 70 5 k = al() işlemi ile en üstteki 99 çekilir.	Yeni Veriler Eklendi koy(44); koy(33); yi → 33 44 70 5 33 ve 44 yeni veriler eklenir.
YIĞININ GÖRSEL NETLİĞİ VE LIFO MANTIK ŞEMASI			
State: [99, 70, 5] [99] <-- En Üst (yi) [70] [5] Yeni veri üstten eklenir (99 en son geldi).		State: [33, 44, 70, 5] [33] <-- Yeni En Üst (yi) [44] [70] [5] Son giren 33, en üstte (LIFO). 99 nerede? Alındı.	

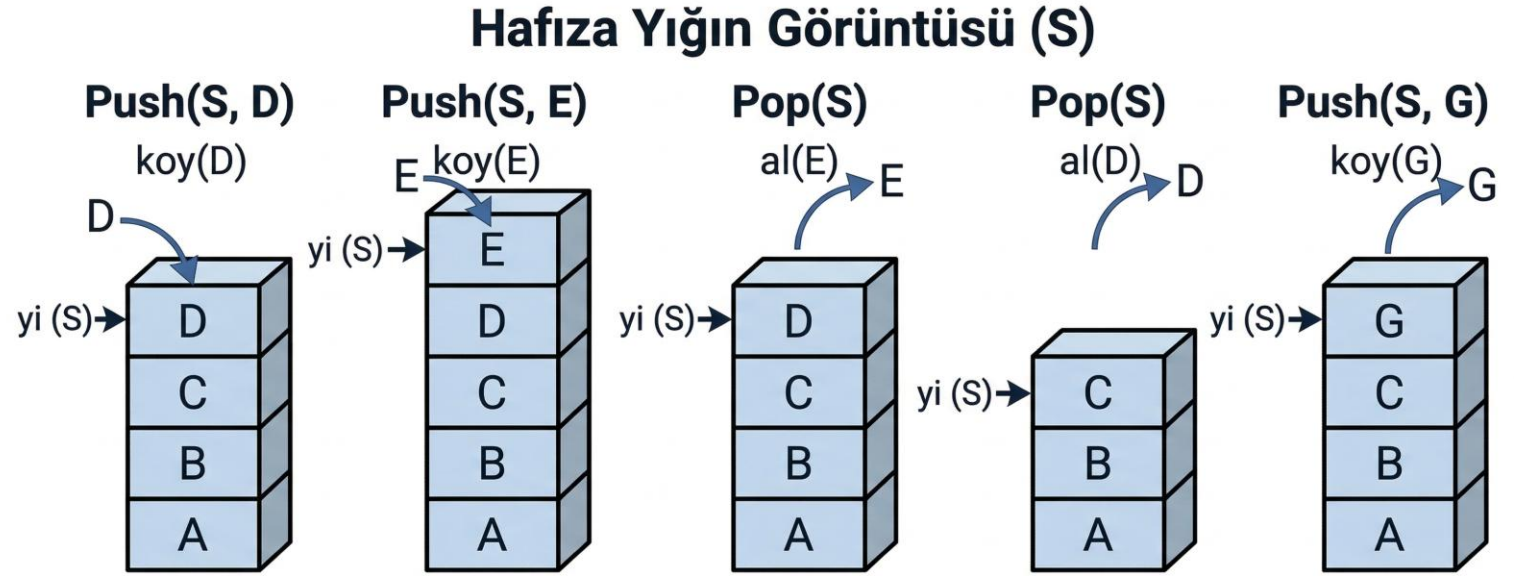
Yığıt veri yapısı son giren ilk çıkar (Last in first out (LIFO)) kuralı ile erişimin yapıldığı, **ara elemanlarına erişilemeyen bir veri yapısıdır**. Elemanın eklendiği ve çıkarıldığı noktaya yığıtın tepesi, ilk elemanın eklendiği en derin noktaya ise yığıtın tabanı denir. Yığıt, eleman ekleme ve eleman silmenin yalnızca bir uçtan yapılabildiği sıralı erişimli veri yapılarıdır.

Yığıt Üzerinde Temel İşlemler

Push->Stack içerisinde bir bilgi koymaya (Stack'in en tepesine koyar)

Pop->Stack içerisinde bir bilgi almaya (Stack'in en tepesinden alır)

Top-> Stack'in en tepesindeki bilgiyi alır ancak stackten çıkartmaz sadece okur.



Dizilerden farklı olarak **yığıtta tutulacak eleman sayısı** üzerine bir **üst limit yoktur(dinamik yazıldı ise)**. Çünkü yığıtın tanımında böyle bir özelleştirme yoktur. Bu yüzden de **yığıta eleman eklerken daha geniş bir yığıta ulaşmak üzere push** fonksiyonu çalışır. Fakat **yığıttan eleman çıkaracağımız zaman** eğer **yığıtta eleman yoksa pop** fonksiyonu çalışmaz. Yığıttan eleman sileceğimiz zaman **yığıtın boş olup olmadığının kontrol edilmesi** gereklidir.

Yığıt üzerinde yapabileceğimiz diğer bir temel işlem yığıtın tepesindeki elemanın ne olduğunu belirlemek olabilir. Yığıtın boş olması halinde bu fonksiyon çalışmayacaktır.

Yığın Dizi Veri Yapısı ile Gerçekleştirilmesi

```
#include <iostream>
using namespace std;
#define SIZE 5
```

. (nokta) operatörü:

- Eğer struct doğrudan tanımlı bir değişkense (pointer değilse) kullanılır.
- > (ok) operatörü:Eğer struct bir işaretçi (pointer) ise kullanılır.

```
struct stack{
    int data[SIZE];
    int top=-1;
```

```
};
stack nstack;
```

```
void pop(){
```

```
    if(nstack.top== -1){
        cout<<"Cikarilacak eleman yok, stack bos"<<endl;
```

```
    }
    else{
        cout<<"Bellekten "<<nstack.data[nstack.top]<<"degeri cikariliyor!!"<<endl;
        nstack.top--;
```

Yığın sıfırdan indekslendiği için en fazla SIZE - 1 indeksine kadar dolabilir.

Eğer SIZE kullanırsak, dizinin sınırlarını aşmaya çalışırız ve hata oluşur

```
void push(int ndeger){
```

```
    if(nstack.top==SIZE-1){
        cout<<"Stack doludur veri girisi yapılamaz"<<endl;
```

```
    }
    else{
        nstack.top++;
        nstack.data[nstack.top]=ndeger;
        cout<<"Stack yapısına "<<ndeger<<" eklendi";
```

```
void yazdir(){
```

```
    if(nstack.top== -1){
        cout<<"Stack bos bu nedenle yazdirilacak eleman yoktur"<<endl;
```

```
    }
    else{
        for(int i=nstack.top; i> -1; i--){
            cout<<nstack.data[i]<<endl;
```

```
int main(){
    pop();
    push(10);
    push(20);
    push(30);
    push(330);
    pop();
    yazdir();
}
```

```
C:\Users\Fatma\Desktop\VERİ YAPILARI\stack>stackarr
```

```
Cikarilacak eleman yok, stack bos
```

```
Stack yapısına 10 eklendiStack yapısına 20 eklendiStack yapısına 30 eklendiStack yapısına 330 eklendiBellekten 330degeri cikariliyor!!!
```

```
30
20
10
```

Yığıtın dizi veri yapısı ile gerçekleştirilmesi 2

```
#include <iostream>
using namespace std;
struct stack{
    int top=0;
    int boyut=4;
};
stack nstack;
int * dizi;
void push(int ndeger){
    if(nstack.top>nstack.boyut || nstack.top==nstack.boyut){
        int *dizi2=(int*)malloc(sizeof(int)*nstack.boyut*2);
        for(int i=0; i<nstack.top; i++){
            dizi2[i]=dizi[i];
        }
        free(dizi);
        dizi=dizi2;
        nstack.boyut=nstack.boyut*2;
    }
    dizi[nstack.top++]=ndeger;
}
void pop(){
    if(nstack.top<=(nstack.boyut*(1/4))){
        int *dizi2=(int*)malloc(sizeof(int)*nstack.boyut/2);
        for(int i=0; i<nstack.top; i++){
            dizi2[i]=dizi[i];
        }
        free(dizi);
        dizi=dizi2;
        nstack.boyut=nstack.boyut/2;
    }
    nstack.top--;
}
void yazdir(){
    for(int i=0; i<nstack.top; i++){
        cout<<dizi[i]<<endl;
    }
}
int main(){
    dizi=(int*)malloc(sizeof(int)*4); //4 boyutlu bir dizi
    push(10);
    push(20);
    push(30);
    push(40);
    push(50);
    push(60);
    push(70);
    push(80);
    push(90);
    push(100);
    pop();
    pop(); pop(); pop();
    yazdir();
}
```

Toplam 16 byte yer ayırır

```
C:\Users\Fatma\Desktop\VERİ YAPILARI\stack>g++ -o stack stack.cpp
```

```
C:\Users\Fatma\Desktop\VERİ YAPILARI\stack>stack
```

```
10
20
30
40
50
```

Bellek Tahsisi:

Örneğin, dizi = (int*) malloc(n * sizeof(int));

malloc çağrısı, size kadar baytlık bir bellek bloğu ayırır.

Tahsis edilen bellek işi bittiğinde free() ile serbest bırakılmalıdır. Aksi halde **memory leak (bellek sızıntısı)** oluşur.

Yığıtın Dizi ile Tasarımında,

Bu yöntem verinin geçici olarak tutulacağı yerin bütünüyle bir dizi olarak bildirilmesine dayanır ve tutulacak toplam veri sayısı kadar boş bir bellek alanı bu iş için ayrılır. Örneğin veri türü tam sayı ise n tane tam sayının sığacağı bir bellek alanı dizi bildirimi yapılarak oluşturulur. Fakat dinamik tasarım da mümkündür.

Yığıt bir bakıma **sıralı nesneler topluluğudur** ve C++ dilinde tam bu tanıma uygun **dizi veri yapısı** vardır. Eğer problemi yığıt ile çözmeye karar vermişsek bunu bir dizi ile gerçekleştirebiliriz. Oysa ikisi de gerçekte çok farklı kavramlardır.

Dizilerin eleman sayıları **sabittir**, yığıt ise dinamik bir yapıdır. Bu farka rağmen **bir yığıt dizi üzerinde tanımlanabilir**. Burada yığıtı idare edecek **kapasitede bir dizi tanımlar ve bu dizinin sadece bir ucundan işlem yapabilirsek** diziyi yığıt gibi kullanabiliriz. Bu durumda **bir diziye ve izlenecek bir indise** ihtiyaç vardır.

Yığıt, dizi üzerinde gerçekleştirildiği için **dizi sabit boyutlu** bir veri yapısıdır. Dizi dolduğu durumda **yığıtın işleyişine devam edebilmesi için dizinin genişletilmesi gerekir**. Yığıt elemanlarının tipi **tam sayı olmak zorunda değildir**. Kayan noktalı ya da karakter tiplerinde de olabilirler.

Normal şartlar altında push içinde boş kontrolü yapmana gerek yok.

YIĞIT MANTIĞI İLE YIĞIT TASARIMI

Eğer tepe == NULL ise
nnode->next = NULL olur yani yeni
düğüm tek başına yığın olur

tepe → [30 | *] → [20 | *] → [10 | NULL]

```
#include <iostream>
using namespace std;

struct Node {
    int veri;
    struct Node* next;
};

Node* push(Node* tepe, int eklenen) {
    Node* nnode = new Node();
    nnode->veri = eklenen;

    if (tepe == NULL) {
        // Yığın boşsa
        nnode->next = NULL;
        tepe = nnode;
    } else {
        // Yığın doluysa
        nnode->next = tepe;
        tepe = nnode;
    }

    cout << eklenen << " yigina eklendi (Yeni Tepe)." << endl;
    return tepe;
}

Node* pop(Node* tepe) {
    if (tepe == NULL) {
        cout << "Hata: Yigin bos, cikarilacak eleman yok!" << endl;
        return NULL;
    }

    Node* temp = tepe;    // Silinecek olanı yedekle
    tepe = tepe->next;    // Tepeyi bir altındaki elemana kaydır

    cout << temp->veri << " yigindan cikarildi." << endl;
    delete temp;          // Belleği temizle

    return tepe;
}
```

```
void yazdir(Node* tepe) {
    if (tepe == NULL) {
        cout << "Yigin bos." << endl;
        return;
    }
    Node* iter = tepe;
    cout << "Yigin Durumu (Yukardan Asagi):" << endl;
    while (iter != NULL) {
        cout << "|" << iter->veri << " |" << endl;
        iter = iter->next;
    }
    cout << " ----" << endl;
}

int main() {
    Node* tepe = NULL;

    tepe = push(tepe, 10);
    tepe = push(tepe, 20);
    tepe = push(tepe, 30);

    yazdir(tepe);

    tepe = pop(tepe);
    yazdir(tepe);

    return 0;
}
```

Gerçek bir **Stack (Yığın)** yapısının en önemli özelliği, tüm işlemlerin (ekleme ve silme) listenin sadece **bir ucundan** yapılmasıdır. Bu sayede hiçbir döngüye (while) ihtiyaç duymadan, çok hızlı bir şekilde işlem yapılır.)

Yığıt İle Problem Çözümü

Şimdi bir örnek problemi ele alıp temel işlemleri tanımlanan yığıt veri yapısı ile örnek problemin çözüp yöntemi olarak nasıl kullanabileceğini inceleyelim.

$a-(3+(b*(c+d)-(k-3)/(c+x)+d)*(a+(b+c)-2)$ ifadesini parantez yönü ile incelersek hangi kurallara göre karar verdiğimizi hatırlayalım.

1. Kural = eşit sayıda sağ ve sol parantez olması ifadenin sonucunu doğru hesaplanması bakımından önemlidir.
2. Kural = bir sağ parantezden önce mutlaka bir sol parantez olmalıdır.
Yani bir parantezin kapatılabilmesi için mutlaka daha önce açılmış olması gerekmektedir.

$(c+d)-(k-3/(c+x))$ ifadesinde açılan parantez sayısı kapanan parantez sayısına eşit değildir.

$(c+x)+d)*(a+b)/(b+c)-2)$ ifadesinde açılmadığı halde kapanan parantezler vardır ve ikinci kurala göre doğru yazılmamıştır.

Şimdi bu iki kuralın uygulanmasını farklı bir örnek üzerinde test edelim.

```
a - ( 3 + ( b * ( c + d ) - ( k - 3 ) / ( c + x ) + d ) * ( a + ( b + c ) - 2 ) + 5 * c )
0 0 1 1 1 2 2 2 3 3 3 3 2 2 3 3 3 3 2 2 3 3 3 3 2 2 3 3 3 3 2 2 2 1 1 2 2 2 3 3 3 3 2 2 2 1 1 1 1 1 0
```

İfadenin son noktasında $PS=0$ ve hiçbir noktada PS negatif olmadığından ifade **doğru yazılmıştır**.

Pointer to Stack (yığın işaretçisi)

Örnek, Nejat yumuşak, Fatih Adak c/c++ ile veri yapıları ve çözümlü uygulamalar kitabından alınmıştır.

(	c	+	d	)	-	(	k	-	3	/	(	c	+	x	)
1	1	1	1	0	0	1	1	1	1	1	2	2	2	2	1

İfadenin hiçbir noktasında PS negatif değer almamıştır fakat ifade sonunda PS=0 da olmamıştır. PS=1 olması bir tane fazladan açılan yuva olduğunu, fazladan bir sol parantez açıldığı anlamına gelmektedir. Bu ifade, parantez yönü ile geçerli bir ifade **değildir**.

Örnek, Nejat yumuşak, Fatih Adak c/c++ ile veri yapıları ve çözümlü uygulamalar kitabından alınmıştır.

$(c+x)+d)*(a+b)/(b+c)-2$ ifadesine bakıldığında ise farklı bir durum söz konusudur.

(	c	+	x	)	+	d	)	*	(	a	+	b	)	/	(	b	+	c	)	-	2	)
1	1	1	1	0	0	0	-	-	0	0	0	0	-	-	0	0	0	0	-	-	-	-
							1	1					1	1					1	1	1	2

Bu ifadede bazı noktalarda PS negatif değerlerle karşılaşmıştır. Bu, iki noktada açılmamış parantezler kapatılmaya çalışıldığı (PS=-2) anlamına gelmektedir. Bu durum ikinci kurala aykırı olduğundan ifade geçerli değildir.

Aşağıdaki ifade de ifadenin sonunda PS=0 olduğu halde bazı noktalarda negatif değer aldığından **geçersiz** olacaktır.

(	a	+	b	)	)	-	(	c	+	d
1	1	1	1	0	-1	-1	0	0	0	0

Örnek, Nejat yumuşak, Fatih Adak c/c++ ile veri yapıları ve çözümlü uygulamalar kitabından alınmıştır.

SORU!

Şimdi $a-(3+(b*(c+d)-(k-3)/(c+x)+d)*(a+(b+c)-2)$ denkleminde açılan ve kapanan parantez sayılarının eşit olup olmadığını kontrol eden bir program yazmak istiyorum. Nasıl yazacağımızı tartışalım.

(Derste bu sorunun uygulaması yapılacaktır.)

Yığıtın Kullanım Yerleri

Yığıt veri yapısı bilgisayar ve bilişim bilimlerinin pek çok alanında problemlerin doğası gereği en uygun çözüm olmaktadır Örneğin

- Yazılım uygulamalarındaki **Undo** işlemleri yığıt veri yapısı ile en kolay biçimde gerçekleşmektedir. **En son yaptığımız işlemin geri alınması** tam olarak yığıt kullanımını tanımlamaktadır.
- Web browser'lardaki Back butonu** ile en son ziyaret edilen sayfaya ilk önce geri dönme işlemi yığıt veri yapısı ile gerçekleşmektedir.
- Yüksek seviyeli bir dilde yazılmış matematiksel ifadelerin makine kodundan değerlendirilmesi ve işlenmesi** de yığıt veri yapısı ile kullanılmaktadır. Bu ifade **derleyici** tarafından **ters Polonya gösterimine (Postfix Notation)** çevrilir ve Bu ifadeyi değerlendirmek için **yığıt (stack)** kullanılır.
- Yazım kontrolündeki **parantezlerin kontrolünde** yığıt kullanılır. Parantez kontrolü işleminin doğası yığıt veri yapısının kullanımına çok uygundur.
- Program içinden program çağırma işleminin** bulunduğu durumlarda da Yığıt kullanımı kaçınılmaz bir çözümdür. **Birbirini çağırان birçok program olduğunda bu işler eğer yığıt kullanmıyorsak çok karmaşık olabilir idi.**

Özyineli (rekürsif) fonksiyonlar, iç içe alt program yapılarında olduğu gibi, kendi kendini çağırabilen yapılardır. Bu tür fonksiyonlarda her çağrı sırasında, fonksiyona ait parametreler, yerel değişkenler ve dönüş adresi gibi bilgiler sistem tarafından yığıt (stack) veri yapısında saklanır. Böylece her bir çağrıya ait yürütme bağlamı korunarak, fonksiyonun doğru ve düzenli bir şekilde çalışması sağlanır.

KAYNAKÇA

Veri Yapıları ve Algoritmalar, 12. Baskı, Rifat Çölkesen

Nejat Yumuşak, Fatih Adak Veri C++ ile veri yapıları, 3.baskı

C++ ile programlama, Dokuzuncu baskıdan çeviri, Palme Yayınları, Pau Deitel, Harvey Deitel, Çeviri Editörü Cemil Öz

Veri Yapıları, Hakan Kutucu

Algoritma ve Programlamaya giriş, Fahri Vatansever

Programlama ve Veri Yapılarına Giriş C,C++ ve Java ile, Sadi Evren Şeker

<https://chatgpt.com/>

<https://gemini.google.com/>